I would write a boundary-condition test only if I had reason to think that my implementation might fail in a boundary condition. That is quite possible for dates and times, and often if I'm dealing with SQL date ranges versus Ruby date ranges or if time zones are involved, I add tests near the boundary to attempt to break the implementation and catch an off-by-one error. I wouldn't write a series of tests for every length of time completed one day through six months ago, since I would expect all those tests to pass. The resulting Task class looks like this:

basics/07/app/models/task.rb

```
class Task
  attr_accessor :size, :completed_at

  def initialize(options = {})
    mark_completed(options[:completed_at]) if options[:completed_at]
    @size = options[:size]
  end

  def mark_completed(date = Time.current)
    @completed_at = date
  end

  def complete?
    completed_at.present?
  end

  def part_of_velocity?
    return false unless complete?
    completed_at > 21.days.ago
  end

  def points_toward_velocity
    part_of_velocity? ? size : 0
  end
end
```

Using the Time Data

With the task tests passing, it's time to switch your attention back to the Project test. You need to make a slight tweak to your project_spec setup so that you have tasks that are in and out of the three-week velocity window:

basics/07/spec/models/project_spec.rb

```
let(:project) { Project.new }
let(:newly_done) { Task.new(size: 3, completed_at: 1.day.ago) }
let(:old_done) { Task.new(size: 2, completed_at: 6.months.ago) }
let(:small_not_done) { Task.new(size: 1) }
let(:large_not_done) { Task.new(size: 4) }

before(:example) do
  project.tasks = [newly_done, old_done, small_not_done, large_not_done]
end
```